Project Overview and Objectives

Project Name: Production-Grade Automated Weekly Newsletter Generator
Objective: Upgrade a functional but fragile weekly newsletter automation into a robust, production-grade system that is fault-tolerant, auditable, self-healing, and human-centric.

The original Week 8 workflow fetched news from five Kenyan sources, extracted headlines via AI, merged them, and generated a newsletter intro. However, it had weaknesses: no fallback when the primary AI failed, no validation of AI outputs, no detailed logging, and no way to answer “What happened last Friday?”.

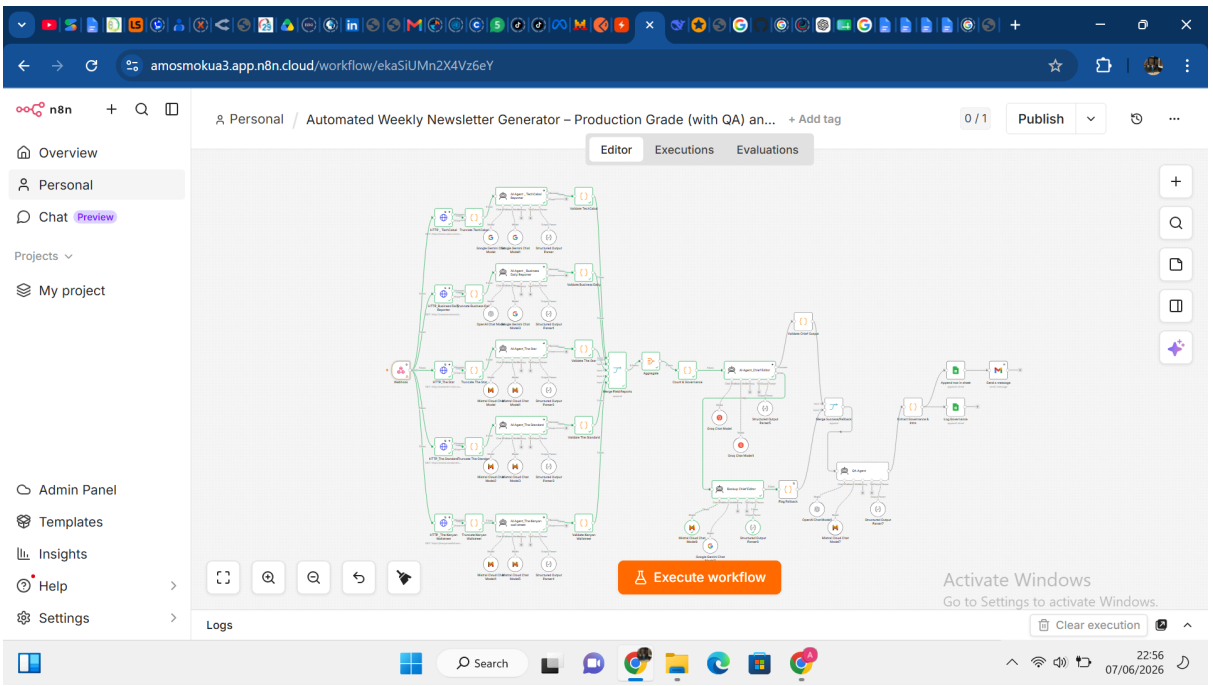
This capstone addresses those gaps by implementing:

- Structured AI output and JSON reliability – Strict output parsers and defensive validation.
- Fault tolerance and resilient design – Truncation, continue-on-error, backup AI, and global error handling.
- Model selection, constraints and fallbacks – Primary (llama-3.3-70b-versatile) and fallback (magistral-small-latest) models, with orange-error-output routing.
- Operational logging and governance – A full Governance sheet and an audit workflow that answers any date’s execution in plain English.

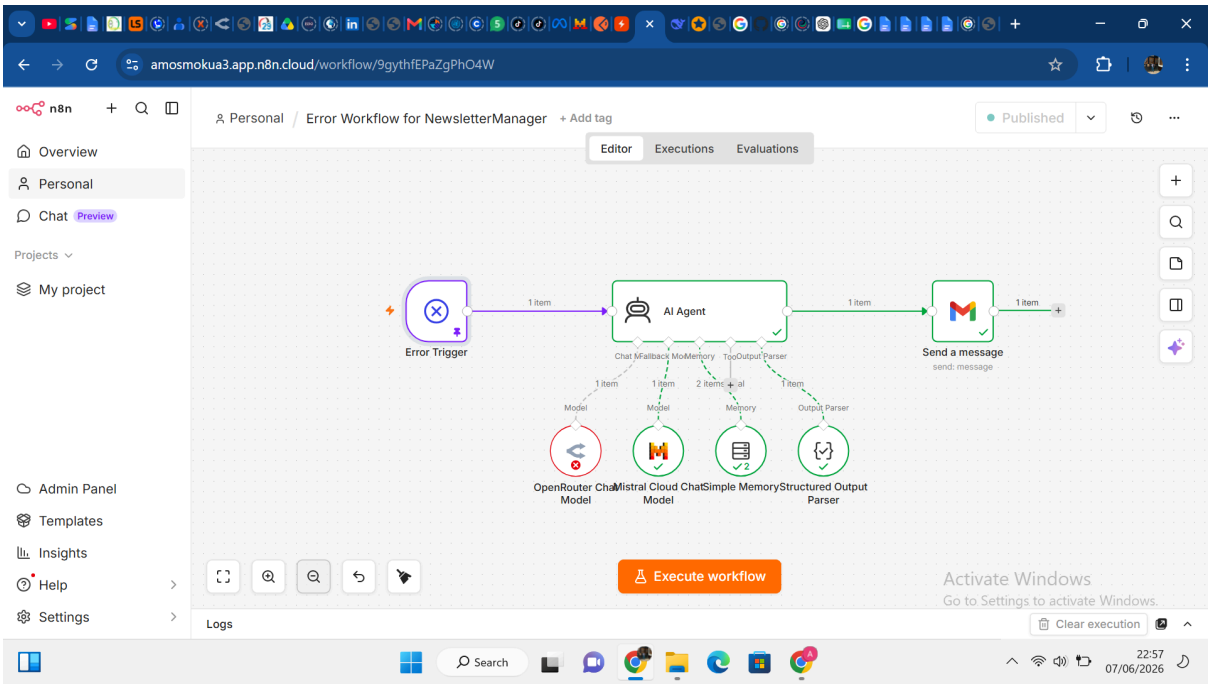
Additionally, we added a Quality Assurance (QA) agent that validates the intro against business rules (length, slang, Markdown, relevance) and generates a weekly parting thought – a unique, human-centric sign-off.

Architecture Diagrams

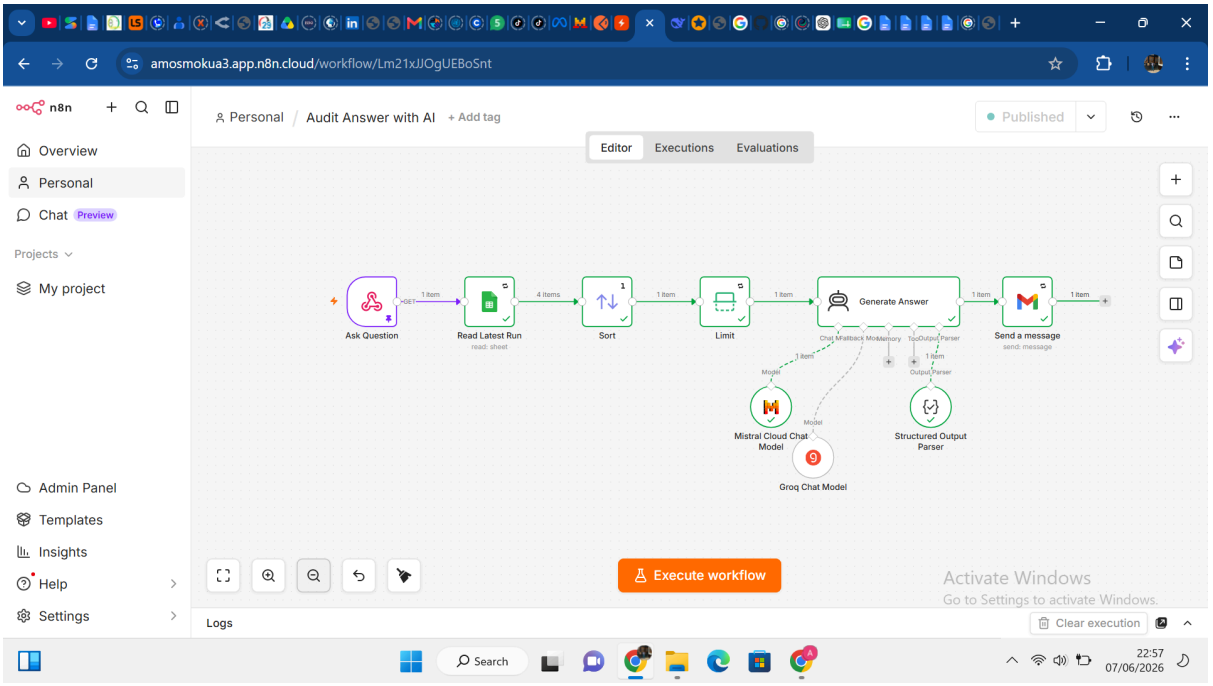
Main Workflow (Newsletter Generator)



Error Workflow (Global Fallback)



Audit Workflow (Answer “What happened last Friday?”)



Workflow Explanations

Main Workflow

1. Trigger – A schedule (Friday 14:00) or webhook starts the workflow.
2. Parallel fetching – Five HTTP Request nodes fetch raw HTML from Kenyan sources (TechCabal, Business Daily, The Star, The Standard, Kenyan Wallstreet). Each has `Continue on Error` and a 10s timeout.
3. Truncation – Code nodes strip HTML tags and limit text to 5000 characters to prevent out-of-memory errors.
4. Field Reporter AI – Each source has its own AI Agent that extracts the most important headline and outputs JSON `{"source": "...", "headline": "..."}`. A structured output parser enforces the schema.
5. Validation – Code nodes check that the AI output contains valid `source` and `headline`. Invalid items are marked `valid: false` so they do not break the merge.
6. Merge & Aggregate – A Merge node (Append, 5 inputs) collects all items, then an Aggregate node combines them into a single array under key `data`.
7. Governance counting – A Code node counts how many sources succeeded (`valid: true`) and creates a `governance` object with timestamp, succeeded count, failed sources list, and workflow version.

8. Chief Editor (primary) – Receives the array of headlines. It synthesises them into a newsletter intro under 200 words, using Kenyan slang and Markdown. Outputs JSON `{"intro": "..."}`.
    - If the primary fails (e.g., API timeout, invalid key), its orange error output triggers the Backup Chief Editor (magistral-small-latest) which generates a very short, simple intro.
    - A `Flag Fallback` node adds `_fallback = true` to the fallback output.
9. Validate Chief Output – A Code node ensures the intro is non-empty; if empty, it inserts a placeholder.
10. Merge Success/Fallback – An Append Merge combines the green output of the primary and the output of the backup (only one will have data).
11. QA Agent – Validates the intro: length <200 words, contains Kenyan slang, valid Markdown, relevance. It corrects issues and also generates a one-sentence parting word.
12. Extract Governance & Intro – A Code node pulls the corrected intro, the governance object, the QA results, and the parting word.
13. Google Sheets (Drafts) – Appends a row with Date, Successful Sources, Draft Body, Model Used.
14. Google Sheets (Governance) – Appends a row with Timestamp, Sources Succeeded, Sources Failed, Model Used (dynamic), Workflow Version, Output Status, Fallback Triggered, QA Passed, QA Issues.
15. Gmail – Sends an HTML email with the newsletter intro and the parting thought.

Error Workflow

- Error Trigger – Activated when the main workflow fails fatally (e.g., an HTTP node set to `Stop Workflow`).
- AI Agent – Receives error metadata: workflow name, failing node, error message. It outputs JSON `{"hypothesis": "...", "action": "..."}`.
- Gmail – Sends an email to the ops team with the hypothesis and recommended action.

Audit Workflow

- Webhook – Accepts a POST request with a JSON body `{"date": "YYYY-MM-DD"}`.
- Google Sheets – Reads all rows from the Governance sheet.
- Sort & Limit – Sorts by Timestamp descending and takes the most recent row for that date (the user can query any date by changing the body).

- AI Agent – Converts the row data into a friendly one-sentence answer: “On 2026-06-01, 4 out of 5 sources succeeded. The Star failed. The model used was llama-3.3-70b-versatile. QA passed. The parting thought was ‘Keep learning, keep growing.’”
- Gmail – Sends that answer to the manager.

Node-by-Node Configuration Details

A. Main Workflow (Automated Weekly Newsletter Generator)

#	Node Name	Node Type	Key Configuration
1	`Webhook`	Webhook	Method: POST; Path: custom; Respond: When last node finishes; Response Data: No response body.
2	`HTTP _ TechCabal` (and 4 other HTTP nodes)	HTTP Request	URL: respective source URL; Timeout: 10000 ms; On Error: Continue (using error output).
3	`Truncate TechCabal` (and 4 others)	Code	Mode: Run Once for Each Item; JavaScript: strip HTML tags, limit to 5000 chars, return `{ json: { data: truncated } }`.

4	`AI Agent _ TechCabal Reporter` (and 4 others)	AI Agent (LangChain)	System Message: extract single most important headline, return JSON `{"source":"...","headline":"..."}`; User Message: `={{ $json.data }}`; Output Parser: Structured Output Parser (schema: `{"source":"string","headline":"string"}`); On Error: Continue (using error output); Language Models: fallback pair (e.g., Gemini & Gemini).
5	`Structured Output Parser` (per Field Reporter)	Output Parser	Input Schema: `{"type":"object","properties":{"source":{"type":"string"},"headline":{"type":"string"}},"required":["source","headline"]}`

6	`Validate TechCabal` (and 4 others)	Code	Mode: Run Once for Each Item; JavaScript: check `item.json.output`, set `valid: false` if missing or invalid, return output with `valid` field.
7	`Merge Field Reports`	Merge	Mode: Append; Number of Inputs: 5.
8	`Aggregate`	Aggregate	Operation: Aggregate All Item Data (produces `{ data: [...] }`).
9	`Count & Governance`	Code	Mode: Run Once for All Items; JavaScript: counts `valid: true` items, builds `governance` object with `timestamp`, `sourcesSucceeded`, `sourcesFailedList`, `workflowVersion`.

10	`AI Agent_Chief Editor` (primary)	AI Agent	System Message: witty editor, under 200 words, Kenyan slang, Markdown; User Message: `Here are the headlines: {{ JSON.stringify($json.data) }}`; Output Parser: Structured Output Parser5 (schema: `{"intro":"string"}`); On Error: Continue (using error output); Language Models: Groq `llama-3.3-70b-versatile` (primary) and Groq `llama-3.1-8b-instant` (fallback).
11	`Structured Output Parser5` (Chief Editor)	Output Parser	Input Schema: `{"$schema":"...","type":"object","properties":{"intro":{"type":"string"}},"required":["intro"]}`

12	`Validate Chief Output`	Code	Mode: Run Once for Each Item; JavaScript: checks `output.intro`, if invalid sets placeholder: `"⚠️ Newsletter generation encountered a technical issue..."`.
13	`Backup Chief Editor`	AI Agent	System Message: backup editor, very short intro (<80 words), basic Kenyan slang, apology; User Message: `={{ JSON.stringify($node["Count & Governance"].json.data) }}`; Output Parser: Structured Output Parser6 (same schema as primary); Language Models: Mistral `magistral-small-latest` (primary) and Google Gemini (fallback).

14	`Structured Output Parser6` (backup)	Output Parser	Same schema as primary (`{"intro":"string"}`).
15	`Flag Fallback`	Code	Mode: Run Once for Each Item; JavaScript: `data._fallback = true; return { json: data };`
16	`Merge Success/Fallback`	Merge	Mode: Append; Number of Inputs: 2 (green output of primary after validation; green output of backup after flag).

17	`QA Agent`	AI Agent	System Message: QA agent, validate length (<200 words), slang, Markdown, relevance; output `passed` (bool), `corrected_intro` (string), `issues` (array), `parting_word` (string); User Message: `Original intro: {{ $json.output.intro }}\nHeadlines: {{ JSON.stringify($node["Count & Governance"].json.data) }}`; Output Parser: Structured Output Parser7 (schema includes all four fields); Language Models: OpenAI `gpt-4o-mini` with `json_object` response format.
18	`Structured Output Parser7` (QA)	Output Parser	Input Schema: `{"type":"object","properties":{"passed":{"type":"boolean"},"corrected_intro"`

			`:{"type":"string"},"issues":{"type":"array","items":{"type":"string"}},"parting_word":{"type":"string"}},"required":["passed","corrected_intro","issues","parting_word"]}`
19	`Extract Governance & Intro`	Code	Mode: Run Once for All Items; JavaScript: merges QA output, governance object, adds `fallbackTriggered` based on `_fallback` flag; returns `{ intro, parting_word, governance, qa_passed, qa_issues }`.

20	`Append row in sheet` (Drafts)	Google Sheets	Operation: Append; Sheet: `Drafts`; Column mapping: `Date = {{ $now }}`, `Successful Sources = {{ $node["Count & Governance"].json.governance.sourcesSucceeded }}`, `Draft Body = {{ $json.intro }}`, `Model Used = "gpt-4o-mini (fallback: llama-3.1-8b-instant)"`

21	`Log Governance`	Google Sheets	Operation: Append; Sheet: `Governance`; Column mapping: `Timestamp = {{ $json.governance.timestamp }}`, `Sources Succeeded = {{ $json.governance.sourcesSucceeded }}`, `Sources Failed = {{ $json.governance.sourcesFailedList }}`, `Model Used = {{ $json.governance.fallbackTriggered ? "magistral-small-latest" : "llama-3.3-70b-versatile" }}`, `Workflow Version = {{ $json.governance.workflowVersion }}`, `Output Status = success`, `Fallback Triggered = {{ $json.governance.fallbackTriggered || false }}`, `QA Passed = {{ $json.qa_passed }}`, `QA Issues = {{`

			`$json.qa_issues.join(', ') }}`
22	`Send a message` (Gmail)	Gmail	To: amosmokua3@gmail.com/mercykihugu484@gmail.com; Subject: `Nairobi Nexus Draft – {{ $now.toFormat('yyyy-MM-dd') }}`; Body (HTML): HTML template with `{{ $node["Extract Governance & Intro"].json.intro }}` and `{{ $node["Extract Governance & Intro"].json.partin g_word }}`.

B. Error Workflow (Newsletter Error Handler)

#	Node Name	Node Type	Key Configuration
1	`Error Trigger`	Error Trigger	No configuration needed.

2	`AI Agent`	AI Agent	System Message: diagnostic assistant, output `hypothesis` (brief root cause) and `action` (recommended next step); User Message: `Workflow: {{ $json.workflow.name }}, Failing Node: {{ $json.execution.lastNodeExecuted }}, Error: {{ $json.execution.error.message }}`; Output Parser: Structured Output Parser (schema `{"hypothesis":"string","action":"string"}`); Language Models: OpenRouter (primary) and Mistral (fallback).
3	`Structured Output Parser`	Output Parser	Input Schema: `{"type":"object","properties":{"hypothesis":{"type":"string"},"action":{"type":"string"}},"required":["hypothesis","action"]}`

4	`Send a message` (Gmail)	Gmail	To: `amosmokua3@gmail.com/mercykihugu484.gmail.com`; Subject: ⚠️ `Nairobi Nexus Newsletter – Workflow Failed at {{ $json.execution.lastNodeExecuted }}`; Body (HTML): `Hypothesis: {{ $json.output.hypothesis }} Action: {{ $json.output.action }}`.

C. Audit Workflow (Audit Answer with AI)

#	Node Name	Node Type	Key Configuration
1	`Ask Question`	Webhook	Method: POST; Path: custom; Respond: When last node finishes; Response Data: No response body.
2	`Read Latest Run`	Google Sheets	Operation: Get Many Rows; Sheet:

			`Governance`; No filter – reads all rows.
3	`Sort`	Sort	Sort Type: Simple; Property Name: `Timestamp`; Order: Descending.
4	`Limit`	Limit	Max items: 1; Keep: first items.
5	`Generate Answer`	AI Agent	System Message: audit assistant, convert governance row into a friendly one-sentence answer; User Message: `Here is the execution log: {{ JSON.stringify($json) }}`; Output Parser: Structured Output Parser (schema `{"answer":"string"}`); Language Models: Mistral (primary) and Groq (fallback).
6	`Structured Output Parser`	Output Parser	Input Schema: `{"type":"object","properties":{"answer":{"`

			`type":"string"}},"required":["answer"]}`
7	`Send a message` (Gmail)	Gmail	To: `amosmokua3@gmail.com/mercykihugu484@gmail.com`; Subject: `Audit Answer: What happened last Friday?`; Body: `{{ $json.output.answer }}`

Step-by-Step Explanation of the Entire Workflow

1. A webhook triggers the main workflow.
2. Five HTTP requests run in parallel, fetching HTML from five Kenyan news sites. If any fails, the branch continues (no data).
3. Each HTML is truncated to 5000 characters and sent to its dedicated Field Reporter AI.
4. Each Field Reporter extracts the most important headline and outputs JSON `{"source":"...","headline":"..."}`.
5. Validation Code nodes check the JSON; invalid headlines are marked `valid: false`.
6. All items (including invalid ones) are merged and aggregated into a single array.
7. The `Count & Governance` Code node counts successful sources and creates a governance object.
8. The array of headlines is sent to the primary Chief Editor AI.
    - If the primary succeeds, it outputs a JSON object with the newsletter intro.
    - If the primary fails, the orange error output triggers the Backup Chief Editor, which outputs a simpler intro. A flag `_fallback` is attached.
9. The output (from primary or backup) is validated – if empty, a placeholder intro is used.
10. The success and fallback paths are merged (only one will contain data).

11. The QA Agent validates the intro against business rules, corrects any issues, and generates a parting word.
12. The `Extract Governance & Intro` Code node combines the corrected intro, governance object, QA results, and parting word into one JSON object.
13. Two Google Sheets nodes append rows: one to the `Drafts` tab (for the newsletter archive) and one to the `Governance` tab (for audit).
14. A Gmail node sends the final newsletter intro and parting word to the marketing team.
15. If any fatal error occurs (e.g., an HTTP node set to stop the workflow), the global error workflow runs and sends a diagnostic email with a hypothesis and recommended action.
16. Separately, a manager can use the audit workflow (via webhook with a date in the JSON body) to receive an email answering what happened on any specific day.

Integrations, APIs, and Tools Used

- n8n Cloud (version 2.21.8) – Workflow automation platform.
- Google Sheets API – Reading/writing to `NewsletterManager` spreadsheet (Drafts and Governance tabs).
- Gmail API – Sending emails for newsletter drafts, error alerts, and audit answers.
- AI Providers:
    - Groq (`llama-3.3-70b-versatile`, `llama-3.1-8b-instant`) – Primary Chief Editor, some Field Reporters.
    - Mistral (`magistral-small-latest`, `magistral-medium-latest`, `mistral-large-latest`) – Backup Chief Editor, Field Reporters.
    - OpenAI (`gpt-4o-mini`) – QA Agent, Audit AI Agent, Error AI Agent.
    - Google Gemini – Some Field Reporters.
    - OpenRouter – For additional model access.
- HTML/JavaScript – Truncation Code nodes, HTML email templates, audit HTML form (provided separately).

Error Handling and Recovery Mechanisms Implemented

- HTTP nodes – `On Error` set to `Continue (using error output)`, preventing a single source failure from killing the whole workflow.
- Truncation Code nodes – Reduce HTML to ≤5000 characters, eliminating out-of-memory errors.

- Field Reporter validation – Invalid AI outputs are marked `valid: false` and excluded from the final count, but they do not break the merge or aggregate nodes.
- Backup Chief Editor – Triggered by the primary's orange error output when the primary fails (API timeout, invalid key, etc.). It produces a very short, simple intro, ensuring the workflow still completes.
- Fallback flag – `_fallback = true` is added to the backup output and later logged in the Governance sheet.
- Validate Chief Output – If the intro is empty or invalid, a placeholder text is inserted, preventing downstream failures.
- QA Agent – Automatically corrects common issues (e.g., truncates long intros, adds missing slang, repairs Markdown).
- Global error workflow – Catches fatal errors (e.g., a Code node throwing an unhandled exception) and sends a diagnostic email with a hypothesis and action.
- Governance logging – Every run records which sources succeeded/failed, which model was used, whether fallback was triggered, QA results, and the parting word – making the system fully auditable.
- Audit workflow – Allows a non-technical manager to query any date's execution and receive a plain-English answer, closing the loop on observability.